

Teste de Software

Trabalho: Detecção de Bad Smells e Refatoração Segura

Davi Aguilar Nunes Oliveira

Belo Horizonte, Novembro de 2025

Contents

1	Introdução	2
2	Rede de Segurança e Preparação	2
3	Análise Manual de Bad Smells	2
4	Detecção Automática com ESLint + SonarJS	3
5	Refatoração Segura	4
5.1	Estratégia	4
5.2	Antes e Depois	4
5.3	Validação Contínua	5
6	Validação Final	5
7	Conclusão	5

1 Introdução

O presente relatório descreve o processo completo de detecção e correção de *Bad Smells* no projeto `bad-smells-js-refactoring`. A atividade foi conduzida utilizando uma suíte de testes pré-existente como rede de segurança, análise manual e automática com `ESLint` + `eslint-plugin-sonarjs`, além de refatorações cuidadosas em um novo arquivo de produção.

2 Rede de Segurança e Preparação

Ambiente

- Clonagem do repositório: `https://github.com/CleitonSilvaT/bad-smells-js-refactoring`
- Instalação de dependências: `npm install`
- Ajuste no script de testes para suportar ESM: `node --experimental-vm-modules ./node_modules/jest/bin/jest.js`
- Execução da suíte original: `npm test` → todos os 5 testes passaram.

Esse passo garantiu a linha de base funcional do projeto antes de qualquer alteração na lógica de produção.

3 Análise Manual de Bad Smells

A leitura do arquivo `src/ReportGenerator.js` revelou múltiplos smells clássicos:

1. **Método Longo / Alta Complexidade:** `generateReport` concentra tarefas de preparação, filtragem, formatação e somatório em um único bloco, dificultando a compreensão e invalidando o princípio de responsabilidade única.
2. **Condicionais Aninhadas:** Há cascatas de `if/else` combinando papéis de usuário e formatos de saída (CSV/HTML), tornando o fluxo difícil de seguir e propenso a erros.
3. **Duplicação de Código:** A lógica de inclusão de itens e de cálculo de total aparece repetida para cada formato e perfil, aumentando custo de manutenção.

Esses problemas comprometeram legibilidade, testabilidade e evolução do módulo.

4 Detecção Automática com ESLint + SonarJS

Configuração

Foi criado o arquivo `.eslintrc.json` com o conteúdo recomendado:

- `extends: eslint:recommended` e `plugin:sonarjs/recommended`
- Regras adicionais: `"sonarjs/cognitive-complexity": ["error", 15],`
`"sonarjs/no-identical-functions": "error"`

Resultados

A execução de `npx eslint src/` sobre o código original acusou:

- Violação de complexidade cognitiva: valor 27, acima do limite de 15 (`sonarjs/cognitive-complexity`).
- `if` colapsável (`sonarjs/no-collapsible-if`), reforçando a análise manual das condicionais aninhadas.

Print da execução:

```
❶ $ npx eslint src/
C:\Projetos PUC\bad-smells-js-refactoring\src\ReportGenerator.js
 11:3  error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
 43:14 error  Merge this if statement with the nested one  sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)

❷ $ npm test
> bad-smells-js-refactoring@1.0.0 test
> node --experimental-vm-modules ./node_modules/jest/bin/jest.js

(node:46332) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS tests/ReportGenerator.refactored.test.js
PASS tests/ReportGenerator.test.js

Test Suites: 2 passed, 2 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        0.744 s, estimated 1 s
Ran all test suites.

❸ $ npx eslint src/ReportGenerator.refactored.js
❹ $ git status -sb
## main...origin/main
 M .eslintrc.json
 M package.json
 M src/ReportGenerator.js
 ?? package-lock.json
 ?? src/ReportGenerator.refactored.js
 ?? tests/ReportGenerator.refactored.test.js

❺ $ npx eslint src/
C:\Projetos PUC\bad-smells-js-refactoring\src\ReportGenerator.js
 11:3  error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
 43:14 error  Merge this if statement with the nested one  sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

Figure 1: Saída do `npx eslint src/` antes da refatoração.

O plugin SonarJS foi crucial para quantificar a complexidade e apontar oportunidades específicas de simplificação que poderiam passar despercebidas na análise manual.

5 Refatoração Segura

5.1 Estratégia

- O arquivo original foi mantido intacto; as mudanças ocorreram em `src/ReportGenerator.refactored.js`.
- A suíte de testes original foi copiada para `tests/ReportGenerator.refactored.test.js`, alterando apenas o *import*.
- Refatorações principais:
 - **Extract Method:** métodos auxiliares para normalizar itens, filtrar por perfil, marcar prioridades e formatar CSV/HTML.
 - **Decompose Conditional:** delegação para métodos `isAdmin`, `isStandardUser` e um dicionário de formataadores (`getFormatter`).
 - **Eliminação de duplicação:** fluxo comum de cálculo de total e inclusão de itens concentrado em funções específicas.

5.2 Antes e Depois

Trecho Original (resumo)

```
// Dentro de generateReport
if (user.role === 'ADMIN') {
  if (item.value > 1000) {
    item.priority = true;
  }
  if (reportType === 'CSV') {
    // lógica CSV
  } else if (reportType === 'HTML') {
    // lógica HTML
  }
} else if (user.role === 'USER') {
  if (item.value <= 500) {
    // lógica análoga duplicada
  }
}
```

Trecho Refatorado (equivalente)

```
const formatter = this.getFormatter(reportType);
const visibleItems = this.getItemsVisibleToUser(user, normalizedItems);
const prioritizedItems = this.markPrioritiesForAdmin(user, visibleItems);
return formatter(user, prioritizedItems, total).trim();
```

A nova versão distribui responsabilidades em métodos menores e reutilizáveis, reduzindo drasticamente a complexidade cognitiva.

5.3 Validação Contínua

Após cada conjunto de mudanças, foi executado `npm test`. A suíte completa (arquivo original + refatorado) passou em todas as execuções, garantindo a preservação do comportamento.

6 Validação Final

- **ESLint no arquivo refatorado:** `npx eslint src/ReportGenerator.refactored.js` não apresentou erros, comprovando a eliminação dos smells apontados.
- **Suíte de testes:** `npm test` com os dois arquivos de testes passou integralmente (10 testes).

A comparação entre a saída do ESLint antes e depois demonstra redução significativa na complexidade e na quantidade de problemas reportados.

7 Conclusão

O trabalho evidenciou a importância da combinação entre análise manual, ferramentas de linting e testes automatizados. A rede de segurança fornecida pelo Jest permitiu refatorações agressivas com confiança. A remoção dos *Bad Smells* resultou em código mais legível, modular e sustentável, reduzindo riscos de regressões futuras e facilitando a evolução do módulo de geração de relatórios.

Repositório com as alterações:

<https://github.com/DaviAguilar/bad-smells-js-refactoring>